

Übung 2: GPU Memory Access Benchmark mit PyOpenCL

GitHub User: anggaxws

8. Juli 2026

1 Einleitung und Zielsetzung

Ziel dieses Projekts war es, den Einfluss verschiedener Speicherzugriffsmuster auf die effektive Speicherbandbreite einer GPU zu untersuchen. Dafür wurde ein Benchmark in Python mit PyOpenCL implementiert. Die eigentliche rechenintensive Arbeit wird dabei auf der GPU durch einen OpenCL-Kernel ausgeführt.

Im Mittelpunkt standen vier Fragen:

- Warum ist *coalesced access* deutlich schneller als strided oder zufälliger Zugriff?
- Wie stark verändert sich die erreichbare Bandbreite bei unterschiedlichen Zugriffsmustern?
- Welchen Einfluss hat die Work-Group-Größe auf die Performance?
- Was unterscheidet die Strategie von GPUs zur Verdeckung von Speicherlatenzen von jener klassischer CPUs?

Die Aufgabe ist besonders relevant für datenparallele Programme, da die reine Anzahl an Threads allein nicht ausreicht, um hohe Leistung zu erzielen. Ebenso wichtig ist die Art, wie diese Threads auf den Speicher zugreifen.

2 Konzeptioneller Hintergrund

2.1 SIMT, Warps und Speicherkoaleszenz

Moderne GPUs arbeiten nach dem Prinzip *SIMT* (*Single Instruction, Multiple Threads*). Viele leichtgewichtige Threads werden zu Gruppen zusammengefasst und möglichst synchron ausgeführt. Greifen benachbarte Threads auf benachbarte Speicheradressen zu, kann die Hardware diese Zugriffe zusammenfassen. Dadurch sinkt die Zahl der notwendigen Speichertransaktionen und die effektive Bandbreite steigt.

Genau dieser Fall liegt bei *coalesced access* vor. Werden die Zugriffe jedoch mit steigender Strideweite immer weiter auseinandergezogen oder sogar zufällig verteilt, müssen mehr einzelne Speichertransaktionen verarbeitet werden. Das verschlechtert Cache-Lokalität und Datendurchsatz.

2.2 Latenzverdeckung auf GPU und CPU

CPUs verstecken Speicherlatenzen hauptsächlich durch Caches, Branch Prediction und Out-of-Order-Ausführung. GPUs verfolgen einen anderen Ansatz: Sie halten eine große Anzahl aktiver Warps bzw. Wavefronts bereit. Wenn eine Gruppe auf Daten aus dem globalen Speicher warten muss, kann der Scheduler sofort zu einer anderen ausführbaren Gruppe wechseln.

Damit wird deutlich, warum GPUs stark davon profitieren, wenn viele Threads aktiv sind und gleichzeitig in einem günstigen Speicherlayout arbeiten. Für GPU-optimierte Programme ist deshalb nicht nur die Parallelisierung wichtig, sondern auch die Speicherorganisation.

3 Umsetzung des Projekts

3.1 Projektstruktur

Das Projekt wurde in mehrere klar getrennte Python-Dateien aufgeteilt:

- `main.py`: Einstiegspunkt, Ablaufsteuerung und Erzeugung der Ergebnisdateien
- `benchmark.py`: Messlogik, Datenaufbereitung und Benchmark-Funktionen
- `kernels.py`: OpenCL-Kernel als Quelltext
- `device_info.py`: Auslesen der Geräteinformationen
- `plot_results.py`: Erzeugung der Diagramme
- `results/`: CSV-Dateien, Geräteinformationen und Plots
- `report/`: schriftliche Dokumentation

3.2 Verwendete Werkzeuge

Für die Implementierung wurden folgende Technologien verwendet:

- Python
- PyOpenCL
- NumPy
- Matplotlib
- Pandas
- OpenCL-Laufzeitumgebung der NVIDIA-GPU

3.3 Ablauf der Durchführung

Der praktische Ablauf des Projekts war wie folgt:

1. Auswahl eines OpenCL-fähigen Geräts
2. Erzeugen eines OpenCL-Kontexts und einer Command Queue mit Profiling
3. Kompilieren des OpenCL-Kernels
4. Erzeugen der Eingabedaten und Indexvektoren für verschiedene Zugriffsmuster
5. Mehrfaches Ausführen des Kernels mit Warm-up-Iterationen
6. Messen der Laufzeit über OpenCL-Profiling-Events
7. Berechnen der effektiven Speicherbandbreite
8. Schreiben der Messwerte in CSV-Dateien
9. Erstellen der Diagramme für die Auswertung

3.4 Kernel-Idee

Statt für jedes Zugriffsmuster einen eigenen Kernel zu schreiben, wurde ein einheitlicher Kernel verwendet. Nur das Indexarray `idx` bestimmt, auf welches Element des Eingabearrays zugegriffen wird. Das macht den Vergleich fair, da sich ausschließlich das Zugriffsmuster ändert.

Listing 1: Verwendeter OpenCL-Kernel

```
__kernel void stream_kernel(
    __global const float *a,
    __global float *b,
    __global const int *idx,
    const float c,
    const int n)
{
    int i = get_global_id(0);

    if (i < n) {
        int j = idx[i];
        b[i] = a[j] * c;
    }
}
```

3.5 Erzeugung der Zugriffsmuster

Die Zugriffsmuster werden über das Indexarray modelliert:

- **Coalesced:** `idx[i] = i`
- **Strided:** `idx[i] = (i * stride) % n`
- **Random:** zufällige Permutation aller Indizes

Dadurch liest der Kernel in allen Fällen denselben Datentyp und schreibt denselben Ausgabepuffer, aber die Speicherlokalität verändert sich.

3.6 Messmethode

Die Laufzeit wurde nicht mit CPU-Timern, sondern über Profiling-Events der OpenCL-Queue bestimmt. Dieses Vorgehen ist genauer, weil nur die tatsächliche Kernel-Laufzeit auf dem Gerät gemessen wird.

Vor jeder Messreihe wurden drei Warm-up-Durchläufe ausgeführt. Anschließend wurde jeder Fall siebenmal gemessen und der Median als robuster Hauptwert verwendet. Zusätzlich wurden Minimum, Maximum und Mittelwert gespeichert.

3.7 Berechnung der effektiven Bandbreite

Pro Element werden im Kernel drei Speichertransfers berücksichtigt:

- ein `int32`-Lesezugriff auf `idx[i]`
- ein `float32`-Lesezugriff auf `a[j]`

- ein `float32`-Schreibzugriff auf `b[i]`

Damit ergibt sich:

$$\text{Bytes pro Element} = 4 + 4 + 4 = 12$$

Für ein Problem der Größe n gilt also:

$$\text{transferierte Bytes} = 12 \cdot n$$

Die effektive Bandbreite wurde berechnet durch:

$$\text{Bandbreite} = \frac{\text{transferierte Bytes}}{\text{Laufzeit}}$$

4 Hardware- und Softwareumgebung

4.1 Verwendete GPU

Die Messungen wurden auf folgendem Gerät durchgeführt:

Merkmal	Wert
Plattform	NVIDIA CUDA
Gerät	NVIDIA GeForce RTX 2060 with Max-Q Design
Hersteller	NVIDIA Corporation
Gerätetyp	GPU
OpenCL C Version	OpenCL C 1.2
Treiberversion	610.74
Compute Units	30
Globaler Speicher	6.442.123.264 Byte $\approx$ 6,00 GiB
Lokaler Speicher	49.152 Byte = 48 KiB
Maximale Work-Group-Größe	1024
Maximale Taktfrequenz	1.185 MHz
Maximale Speicherallokation	1.610.530.816 Byte $\approx$ 1.536 MiB

Tabelle 1: Geräteinformationen aus `device_info.json`

4.2 Bemerkung zur Einordnung

Die RTX 2060 Max-Q ist eine mobile GPU mit solider Rechenleistung, aber nicht mit derselben Speicherleistung wie große Desktop- oder High-End-Beschleuniger. Für diese Übung ist sie dennoch sehr gut geeignet, weil die Unterschiede zwischen günstigen und ungünstigen Zugriffsmustern klar sichtbar werden.

5 Experiment A: Vergleich der Zugriffsmuster

5.1 Versuchsaufbau

Für den ersten Versuch wurde eine Problemgröße von $n = 2^{25} = 33\,554\,432$ Elementen verwendet. Die Work-Group-Größe wurde auf 256 gesetzt. Gemessen wurden:

- coalesced
- stride 2
- stride 4
- stride 8
- stride 16
- stride 32
- random gather

5.2 Messwerte

Zugriffsmuster	Laufzeit in s	Bandbreite in GB/s
Coalesced	0,014741504	27,31
Stride 2	0,019448288	20,70
Stride 4	0,028837792	13,96
Stride 8	0,047064640	8,56
Stride 16	0,083497856	4,82
Stride 32	0,084025344	4,79
Random	0,116318208	3,46

Tabelle 2: Ergebnisse für verschiedene Zugriffsmuster

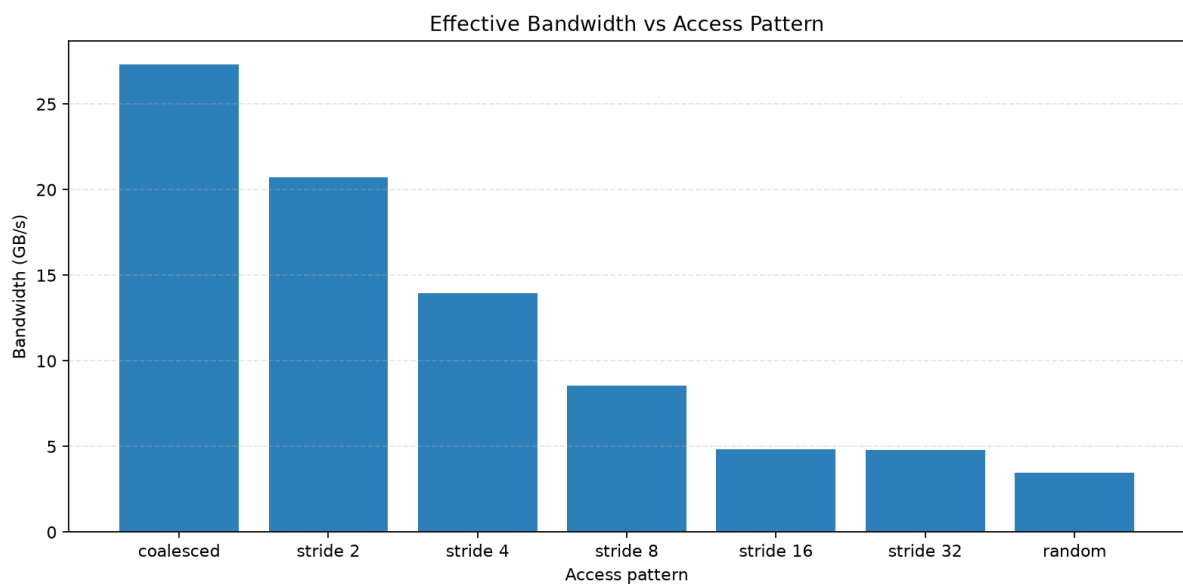


Abbildung 1: Effektive Bandbreite in Abhängigkeit vom Zugriffsmuster

5.3 Interpretation

Die Resultate zeigen sehr deutlich das erwartete Muster:

$$\text{coalesced} > \text{strided} > \text{random}$$

Der beste Wert wurde bei coalesced access mit etwa 27,31 GB/s erreicht. Der zufällige Zugriff fällt dagegen auf nur 3,46 GB/s ab. Damit ist coalesced access ungefähr $27,31/3,46 \approx 7,9$ -mal schneller als random gather.

Auch innerhalb der strided-Zugriffe ist ein klarer Trend erkennbar: Mit zunehmender Strideweite sinkt die Bandbreite stark. Besonders auffällig ist der Übergang von Stride 4 auf Stride 8 und danach auf Stride 16. Ab Stride 16 ist die Lokalisierung bereits so schlecht, dass eine weitere Erhöhung auf Stride 32 kaum noch zusätzliche Verschlechterung bewirkt.

Das bestätigt die zentrale Annahme des Projekts: Für GPU-Code ist nicht nur entscheidend, *dass* parallel auf den Speicher zugegriffen wird, sondern vor allem *wie*.

6 Experiment B: Skalierung mit der Problemgröße

6.1 Versuchsaufbau

Im zweiten Experiment wurde ausschließlich der coalesced-Zugriff betrachtet, um den Einfluss der Problemgröße isoliert zu analysieren. Verwendet wurden die Größen:

$$2^{20}, 2^{22}, 2^{24}, 2^{26}$$

6.2 Messwerte

Elemente n	Laufzeit in s	Bandbreite in GB/s
1.048.576	0,000471040	26,71
4.194.304	0,001865728	26,98
16.777.216	0,007346816	27,40
67.108.864	0,029388800	27,40

Tabelle 3: Skalierung des coalesced-Zugriffs mit der Problemgröße

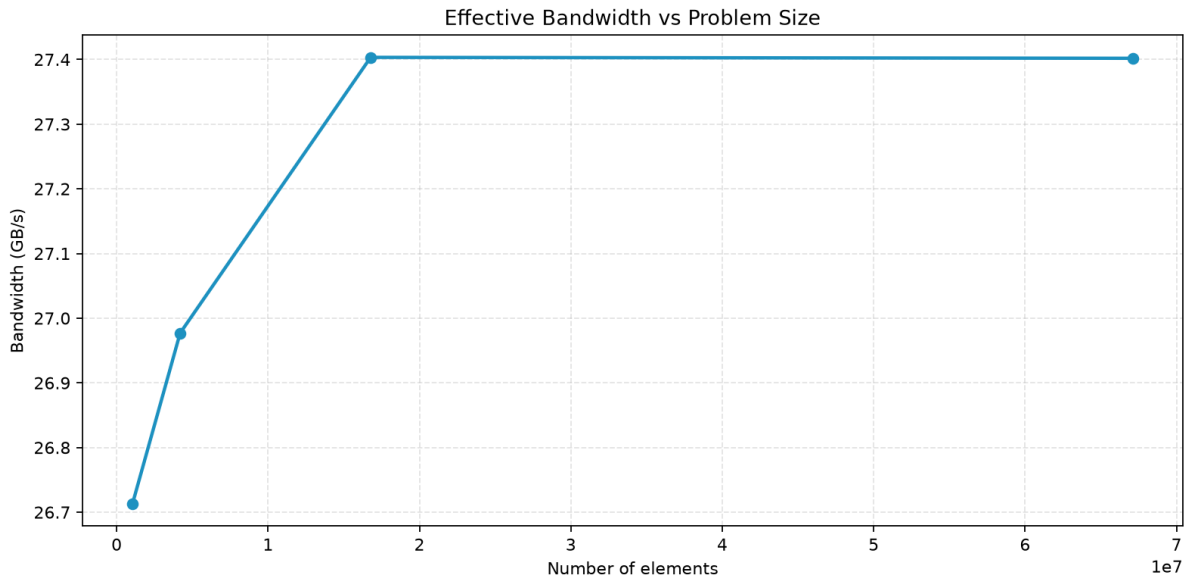


Abbildung 2: Effektive Bandbreite in Abhängigkeit von der Problemgröße

6.3 Interpretation

Die Bandbreite bleibt über alle Größen hinweg erstaunlich stabil im Bereich von etwa 26,7 GB/s bis 27,4 GB/s. Das ist ein gutes Zeichen, weil es darauf hindeutet, dass der Kernel bereits bei der kleineren Problemgröße gut ausgelastet war und die GPU im Wesentlichen in einen stabilen, speicherlimitierten Betrieb gelangt.

Die größte Messung mit 2^{26} Elementen zeigt praktisch keinen Einbruch. Das spricht dafür, dass die Implementierung konsistent arbeitet und die Speichertransfers sauber dominiert werden. Für diesen Benchmark ist das erwartbar, weil die Rechenarbeit pro Element sehr gering ist.

7 Experiment C: Einfluss der Work-Group-Größe

7.1 Versuchsaufbau

Im dritten Experiment wurde wieder die Problemgröße $n = 2^{25}$ verwendet, diesmal mit festem coalesced-Zugriff und variierter Work-Group-Größe. Geplant waren die Größen 32, 64, 128, 256 und 512.

In den tatsächlichen Ergebnissen tauchen jedoch nur 32, 64, 128 und 256 auf. Der Code prüft vor dem Start die vom Gerät und vom Kernel erlaubte maximale Work-Group-Größe und überspringt Werte oberhalb dieses Limits. Das bedeutet, dass für diesen Kernel 512 in der Praxis nicht zulässig war.

7.2 Messwerte

Work-Group-Größe	Laufzeit in s	Bandbreite in GB/s
32	0,014832928	27,15
64	0,014706208	27,38
128	0,014695872	27,40
256	0,014700544	27,39

Tabelle 4: Einfluss der Work-Group-Größe auf die Bandbreite

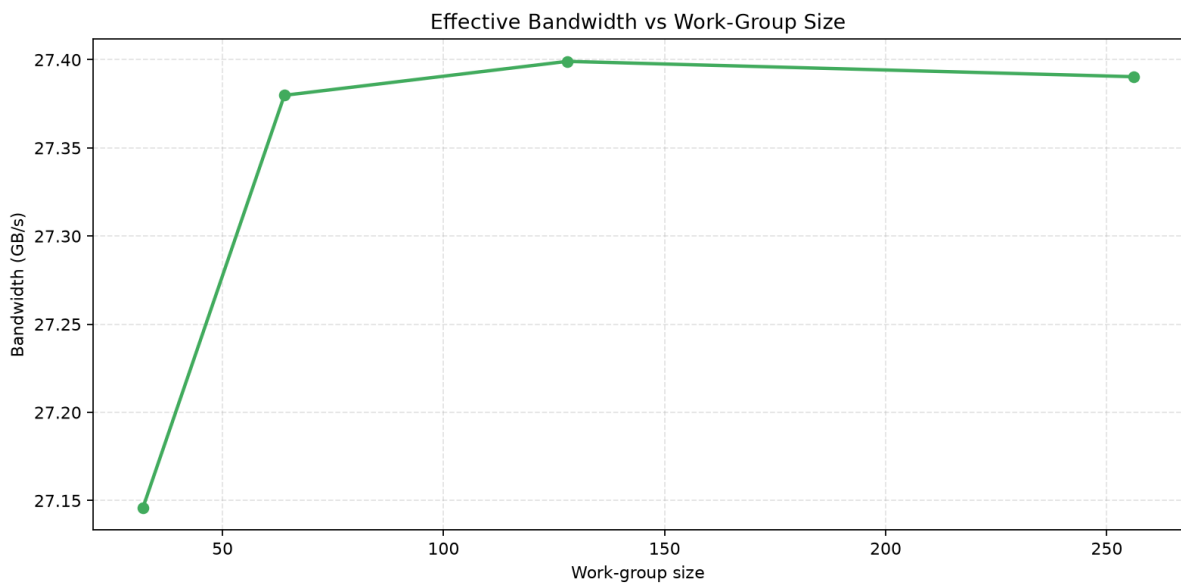


Abbildung 3: Effektive Bandbreite in Abhängigkeit von der Work-Group-Größe

7.3 Interpretation

Der Einfluss der Work-Group-Größe ist in diesem Benchmark vergleichsweise klein. Alle Messwerte liegen sehr eng beieinander zwischen etwa 27,15 GB/s und 27,40 GB/s. Die Differenz zwischen dem schlechtesten und besten Wert beträgt weniger als ein Prozent.

Das lässt sich gut erklären: Der Kernel ist sehr einfach, enthält kaum Kontrollfluss und nur wenig Rechenarbeit. Sobald eine ausreichend sinnvolle Work-Group-Größe erreicht ist, kann die GPU die Speicherlatenz bereits effektiv verdecken. Weitere Vergrößerungen bringen dann nur noch geringe Vorteile.

Trotzdem ist das Experiment nicht bedeutungslos. Es zeigt, dass eine sinnvolle Work-Group-Größe wichtig ist, aber nicht jeder Kernel extrem empfindlich auf diesen Parameter reagiert. In komplexeren Anwendungen mit höherem Registerdruck, mehr Synchronisation oder stärkerer Nutzung des lokalen Speichers könnten die Unterschiede wesentlich größer ausfallen.

8 Zusammenfassung der Ergebnisse

Fragestellung	Beobachtung
Schnellster Zugriff	Coalesced Access mit ca. 27,31 GB/s
Langsamster Zugriff	Random Gather mit ca. 3,46 GB/s
Unterschied bestes/schlechtestes Muster	ungefähr Faktor 7,9
Einfluss der Problemgröße	sehr stabil, ca. 26,7 GB/s bis 27,4 GB/s
Einfluss der Work-Group-Größe	gering, sobald ausreichend groß
Wichtigste Erkenntnis	Speicherlokalität ist für GPU-Performance entscheidend

Tabelle 5: Verdichtete Kernaussagen der Messungen

9 Wie ich das Projekt durchgeführt habe

Aus praktischer Sicht bestand die Arbeit aus zwei Ebenen: zuerst die technische Implementierung und danach die Auswertung der Ergebnisse.

Zunächst habe ich die Benchmark-Idee so umgesetzt, dass alle Zugriffsmuster mit demselben Kernel gemessen werden können. Das war mir wichtig, damit der Vergleich fair bleibt und nicht durch unterschiedliche Kernelstrukturen verfälscht wird. Anschließend habe ich die Messlogik aufgebaut: Warm-up-Läufe, mehrere Wiederholungen, Medianbildung und Ergebnisvalidierung.

Danach habe ich die Ergebnisse in CSV-Dateien exportiert und automatisch Diagramme erzeugt. Dieser Schritt war sehr hilfreich, weil Unterschiede im Zugriffsmuster grafisch viel leichter zu erkennen sind als nur anhand von Zahlenkolonnen. Besonders deutlich sichtbar war dabei, wie stark random access gegenüber coalesced access einbricht.

Für die Dokumentation habe ich dann die Geräteinformationen, Messwerte, Diagramme und theoretischen Hintergründe zusammengeführt, sodass nicht nur die Implementierung, sondern auch die Interpretation nachvollziehbar ist.

10 Meine Erfahrungen bei der Bearbeitung

Ich fand das Projekt besonders lehrreich, weil es einen klassischen Optimierungsgrundsatz der GPU-Programmierung sehr konkret sichtbar macht: Speicherzugriffe sind oft wichtiger als reine Rechenleistung.

Vor der Messung hätte man zwar theoretisch erwarten können, dass coalesced access besser ist, aber die tatsächliche Größenordnung des Unterschieds war trotzdem eindrucksvoll. Dass der random Zugriff fast achtmal langsamer ist als der coalesced Zugriff, zeigt sehr anschaulich, warum Datenlayout und Thread-Zuordnung in GPU-Programmen so entscheidend sind.

Ebenfalls interessant war die Beobachtung, dass die Work-Group-Größe in diesem konkreten Benchmark nur einen kleinen Einfluss hat. Das erinnert daran, dass nicht jeder Performanceparameter in jedem Kontext gleich wichtig ist. Bei diesem sehr einfachen Speicherbenchmark dominiert eindeutig das Zugriffsmuster.

Aus meiner Sicht war die Arbeit mit PyOpenCL angenehm, weil sich damit vergleichsweise schnell eine reproduzierbare Messumgebung aufbauen lässt, ohne direkt ein vollständiges C/C++-OpenCL-Projekt pflegen zu müssen. Gleichzeitig wird aber auch klar, dass man die zugrunde liegende Hardware und die OpenCL-Ausführungslogik verstehen muss, um die Ergebnisse richtig zu deuten.

11 Fazit

Die Übung hat klar gezeigt, dass die effektive Speicherbandbreite einer GPU stark vom Zugriffsmuster abhängt. Coalesced access liefert mit Abstand die beste Performance, während strided und insbesondere zufällige Zugriffe den Datendurchsatz massiv reduzieren.

Die Messungen mit unterschiedlichen Problemgrößen zeigen, dass der Benchmark stabil arbeitet und die GPU bei coalesced access schnell einen konstanten Leistungsbereich erreicht. Die Untersuchung der Work-Group-Größen zeigt außerdem, dass dieser einfache Kernel nach Erreichen einer vernünftigen Größe nur noch wenig von weiterer Variation profitiert.

Die wichtigste Erkenntnis aus dem Projekt ist daher:

Für GPU-Programme ist eine gute Speicherlokalität oft genauso wichtig wie die Parallelisierung selbst.

Wer hohe GPU-Performance erzielen möchte, sollte Daten so anordnen, dass benachbarte Threads möglichst benachbarte Speicherbereiche lesen und schreiben. Genau diese Regel wird durch die Ergebnisse dieser Übung sehr deutlich bestätigt.